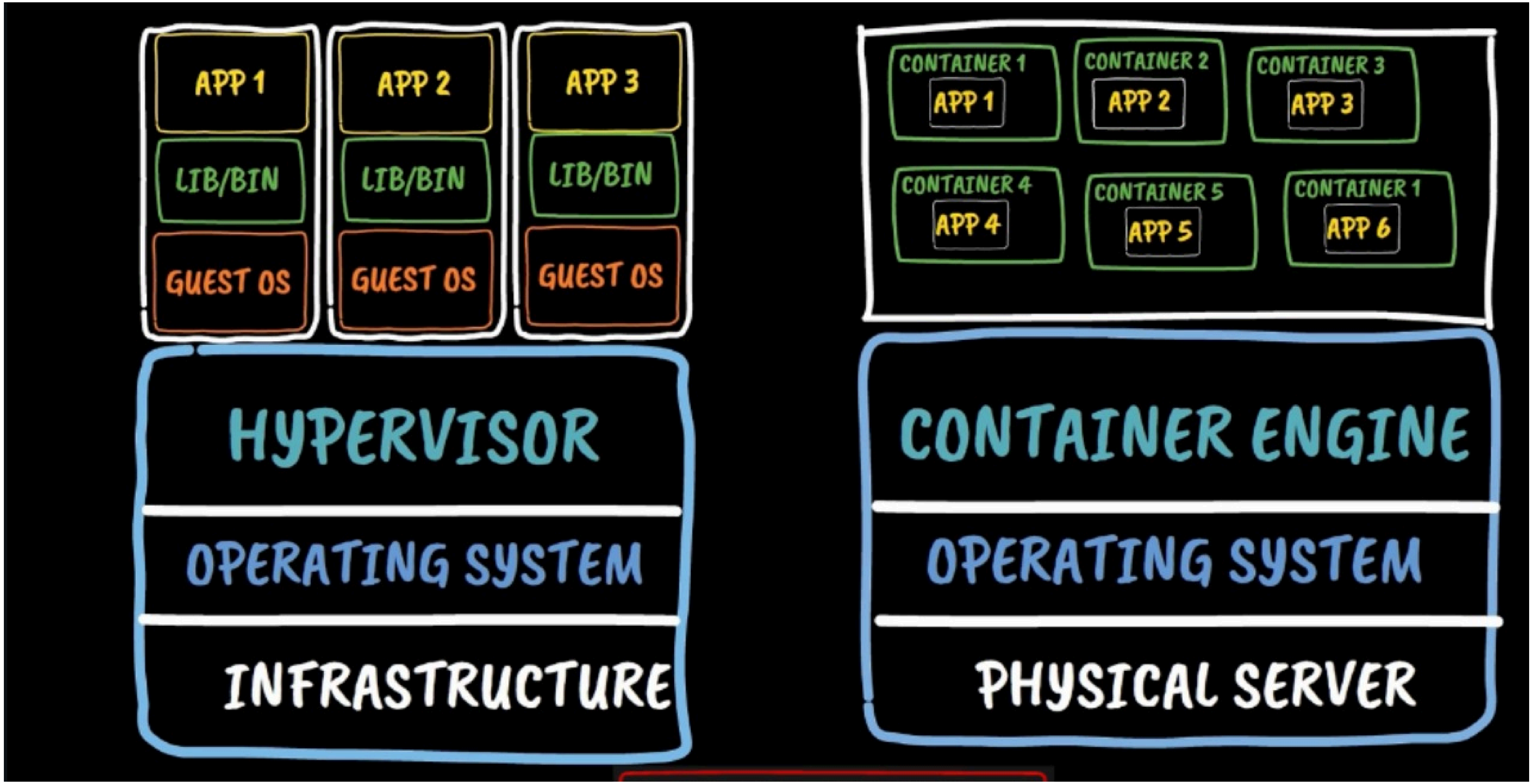
CLOUD COMPUTING TRAINING

DOCKER

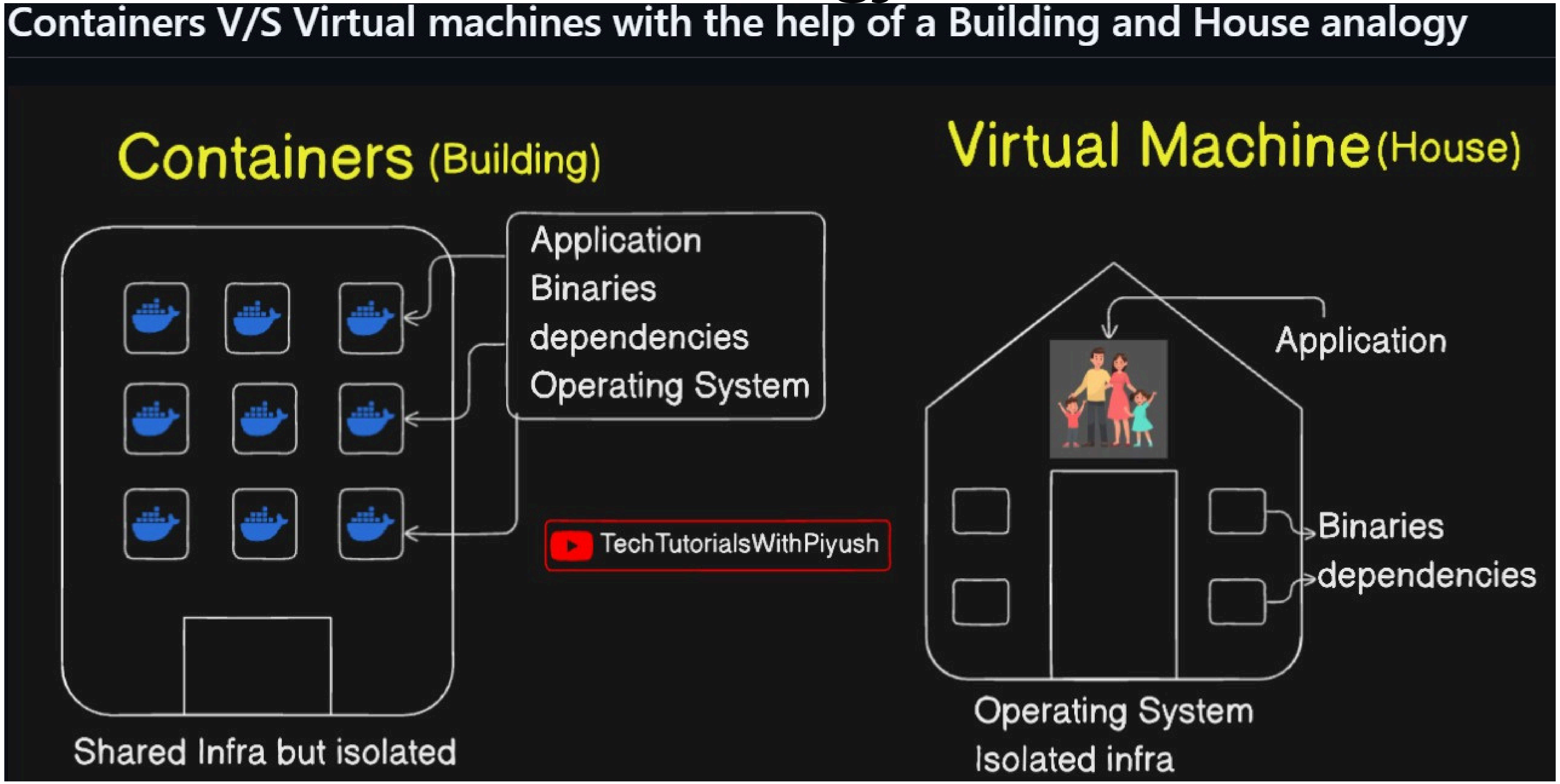
Docker is a platform that creates isolated containers to package an application with all its dependencies so it can build, ship, and run anywhere consistently.

Understanding Container v/s Virtual Machines



Container: A lightweight, isolated environment that shares the host OS kernel to run applications quickly and efficiently.
Virtual Machine (VM): A full virtual computer that includes its own operating system, making it heavier and slower than containers. text

Containers v/s vitual machine with the help of building and house anology



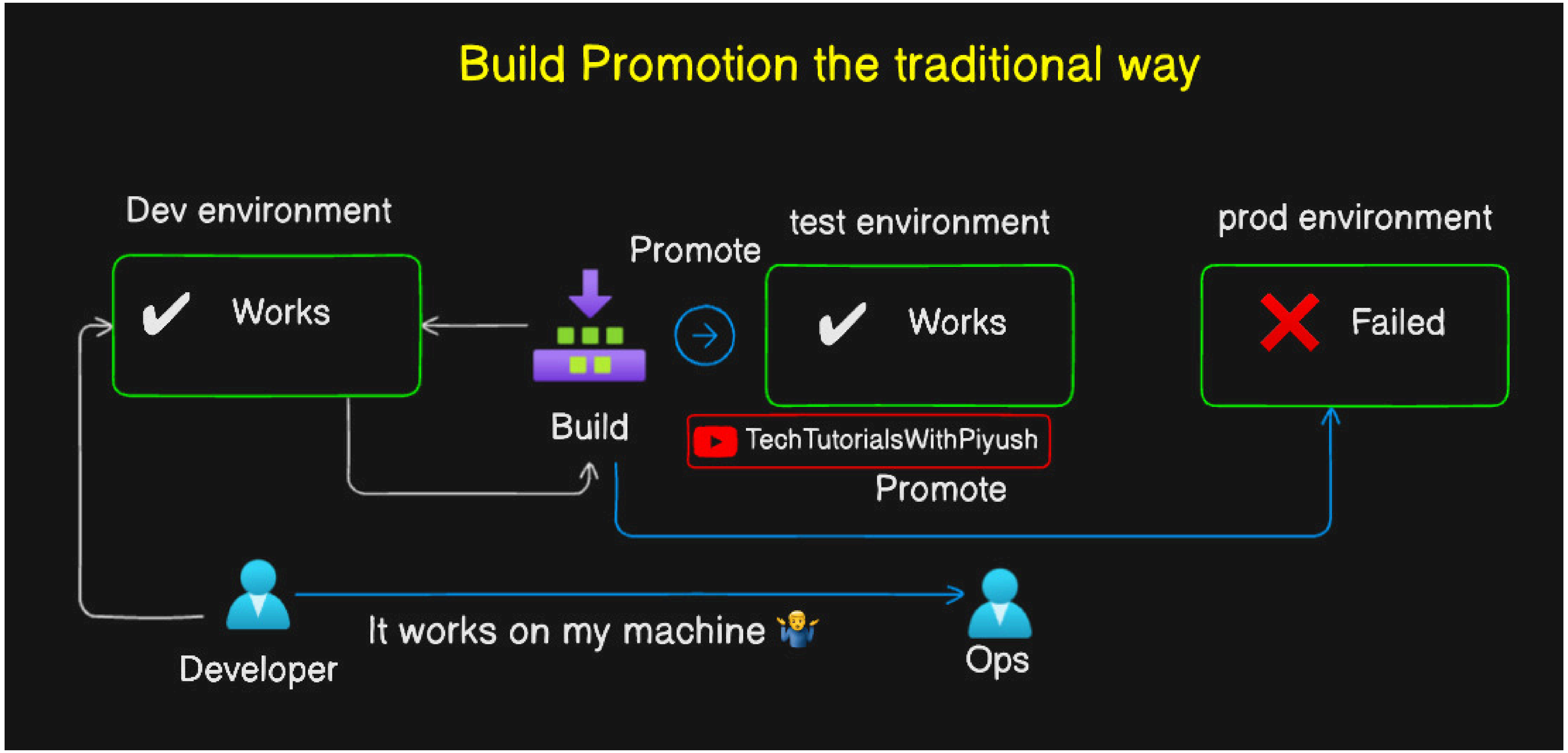
Virtual Machine (VM):

A Virtual Machine is like a separate house, where each house has its own foundation and utilities (operating system). It is fully independent, secure, but uses more resources.

Container:

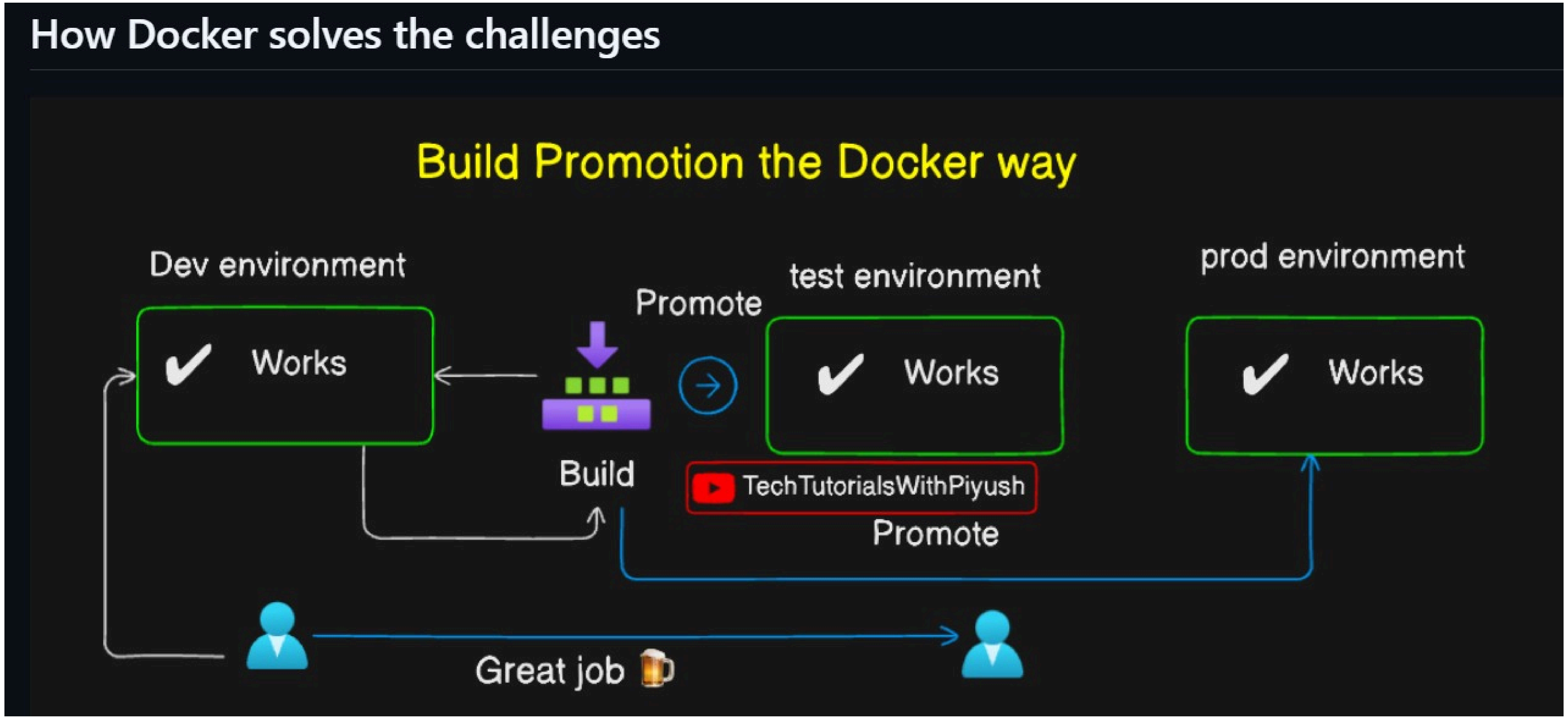
A Container is like an apartment in a building, where all apartments share the same building structure (host operating system). It is lightweight, starts quickly, and uses fewer resources.

Build promotion the traditional way



In the traditional way, developers write code and manually hand it over to the testing team.
After successful testing, the build is manually promoted to the staging environment.
If approved, it is manually deployed to the production environment.
This process is slow, requires manual effort, and is more prone to errors.

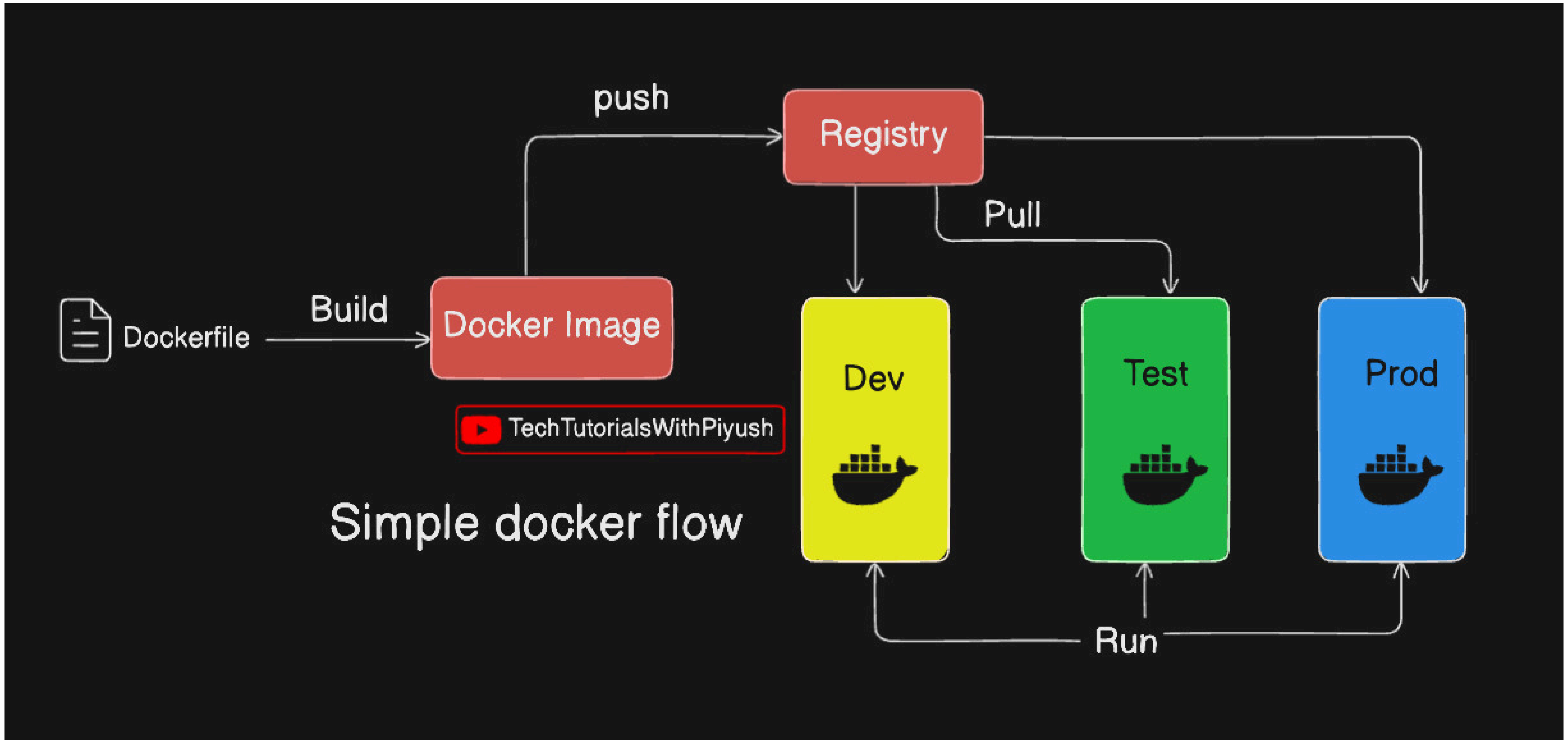
Build promotion the docker way



Build Promotion – Docker Way

With Docker, the application is packaged into a Docker image only once.
The same image is promoted from development to testing, staging, and production.
This ensures consistency across all environments.
It is faster, automated, and reduces deployment errors.

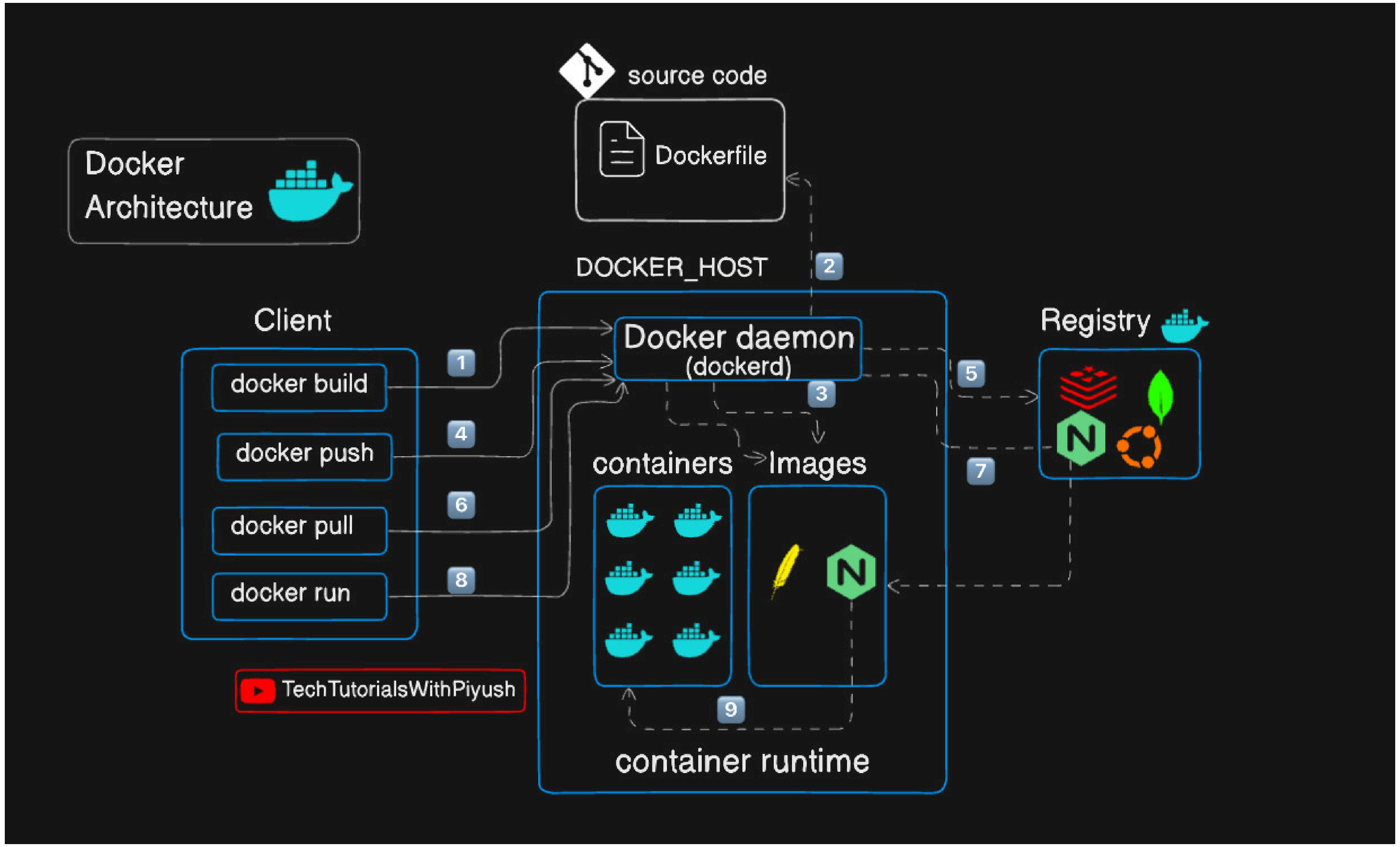
Simple docker flow



Simple Docker Flow:

1. Write the application code.
2. Create a Dockerfile.
3. Build a Docker image (docker build).
4. Run the image as a Docker container (docker run).
5. Test and deploy the same container in any environment (Dev → Test → Production).

Docker architecture



Docker Architecture

1. Docker Client – The user runs Docker commands (e.g., docker build, docker run).

2. Docker Daemon (Docker Engine) – Processes the commands and manages images and containers.

3. Docker Images – Read-only templates used to create containers.

4. Docker Containers – Running instances of Docker images where the application executes.

Flow:
Docker Client → Docker Daemon → Docker Image → Docker Container 🚀